SECURITY · MIGRATION GUIDE

PHP Commerce: Moving Off Shared Hosting

Familiar is not the same as safe. What to move, where to, and in what order — after the October 2026 delistings and checkout hijack.

> **Familiar is not the same as safe.** Protecting customers now takes wholesale change, not patches — especially where personal data (PII), card data (PCI) or health data (PHI) is involved. The October 2026 checkout hijack ran through apps merchants had trusted for years. Shopify and Google have been raising these requirements since 2022. Most of the deadlines have passed; what remains is enforcement — delisting, blocked installs and lost ad eligibility.

Who this is for: merchants, agencies and app developers running WordPress and WooCommerce, Magento or Adobe Commerce, or PHP-backed Shopify apps — loyalty, upsell, product options, wishlists — on shared or single-VM hosting.

The question it answers: what to move, where to, and in what order, so that one breached plugin, one compromised vendor or one runtime flaw cannot reach every customer on the server.

The PHP runtime itself is covered in the companion article: patch for **php2shell** (CVE-2026-109618 and CVE-2026-109619) and keep the Cloudflare WAF rules in Block — Entitlement Security → php2shell. This article is about the architecture around it. For the owner and legal view of the same deadlines — what goes dark, and who carries the liability — see the Risk & Liability Brief: The 2026 Client-Side Cliff.

Vocabulary

TERM	MEANS	IS NOT
Shared VM	One server where many sites share an operating-system user, a database user and a filesystem	A private server with isolated apps
Private VM with Docker	One server the internet cannot connect to, with each application in its own sealed container, reached only through a Cloudflare Tunnel	Anything with an open inbound port
TLS transport	The sealed, tamper-evident envelope data travels in on every hop — shopper to edge, edge to server, server to Xano, server to database	Only the padlock in the browser
Sidecar	A small helper that runs beside each app and does one job — here, asking Xano `/auth/me` who the caller is and attaching a verified answer	The code-injection attack; it closes the hole where an attacker types someone else's customer ID
Real-time data binding	Prices, stock and points read live from the system that owns them	A copy refreshed by a nightly sync
AI task binding	An automated job limited to one tenant, one task and the tools that task needs	An agent holding a platform key
Client-side redirect	A script in the shopper's browser sending them to another domain	A 301 issued by the server

The requirements are not new

SINCE	REQUIREMENT	WHAT IT DEMANDS	REFERENCE
Oct 1, 2022	Shopify protected customer data (API 2022-10)	Personal data redacted by default; apps apply for access and meet Level 1 and Level 2 data-protection requirements	Shopify
Mar 2022, mandatory Mar 31, 2025	PCI DSS v4.0, requirements 6.4.3 and 11.6.1	An authorised inventory of every script on payment pages, integrity checks, and alerts on unauthorised change — the control that catches a checkout hijack	BDO summary
Mar 2024	Google Consent Mode v2 for EEA traffic	Consent signals for ad storage, analytics, ad user data and ad personalisation	Google
Aug 26, 2024	Shopify Customer Account API	At least Level 1 protected-customer-data approval	Shopify
Apr 1, 2025	New public Shopify apps GraphQL-only; REST Admin API legacy	New App Store submissions cannot use REST	Shopify Partners
Jun 30, 2026	Shopify Scripts stop executing	Checkout logic must be Shopify Functions	Shopify changelog
Oct 16, 2026, 15:00 UTC	Shopify API version 2025-10 no longer accessible	Apps still on unsupported resources after an upgrade deadline are delisted	Shopify versioning

October 2026: delistings and the checkout hijack

About **126 apps** were pulled from the Shopify App Store between October 14 and 18, 89 of them on October 15. Most of that number is versioning, not attack: API 2025-10 stopped being accessible on October 16, and delistings are routine — LetsMetrix counted 512 between September 22 and October 21, about 17 a day. Shopify has published no count and named no apps.

Only a few are tied to the hijack. Between October 10 and 20, attackers changed vendor-side settings so that shoppers clicking Checkout were sent to a lookalike payment page. **Shopify's own checkout was not breached.** The code came from vendors' configuration APIs and internal tools, not from the app files Shopify hosts — which is the risk of a third-party server holding store access.

APP	STATUS	BASIS
Hulk Product Options	Hijack confirmed (Oct 10)	Developer statement — cside
Selleasy	Hijack confirmed (Oct 13)	Developer statement — LetsMetrix
Swym Wishlist Plus	Hijack confirmed (about Oct 14)	Unauthorised configuration change — LetsMetrix
Rebuy Engine — PHP app	Delisted; **not** checkout hijacking, self-reported	Statement by James Van Erck, co-founder and COO, on LinkedIn, 22 October 2026: on Oct 14, attackers found an unprotected sign-up path that skipped inbox verification, registered accounts on the company's own email domain, and inherited staff privileges on internal development stores. They read product settings and data-source rules for 11 merchant stores; no edits, no API keys or tokens exposed; merchants notified within 24 hours; contained within hours. Not independently verified
Privy; Boost AI Search & Filter	Delisted Oct 18; link to the hijack unconfirmed	Tracker listings, no vendor confirmation
17TRACK; Consentmo GDPR Compliance	Delisted; link unconfirmed	Tracker listings; Consentmo's rests on a single confidential source

Rebuy's account is the identity half of the same lesson. Nothing in the runtime failed: an email address on the company's domain was treated as proof of being staff, and staff accounts carried standing access to stores. Its remediation — privileges only for a named group, multi-factor authentication for every merchant, a third-party penetration test of the authentication services —

is the permissions model this article argues for: identity verified at the server, never inferred from what the caller typed, and access granted per task rather than held by default.

The same mistake is not unique to apps. Identity providers and the software built on them have shipped it too:

CASE	WHAT WAS TRUSTED	WHAT IT ALLOWED	REFERENCE
Microsoft Entra ID — nOAuth (first disclosed June 2023; still present in about 1 in 10 of 100+ tested SaaS apps in June 2025)	An unverified email claim used as the user's identifier	With only an attacker-controlled Entra tenant and the victim's email address, full takeover of the victim's SaaS account. MFA, conditional access and Zero Trust policies did not stop it	Semperis
Okta AD/LDAP delegated authentication (advisory Nov 1, 2024; introduced July 2024)	A cached earlier login for usernames longer than 52 characters — often email addresses	Sign-in with the username alone, without the right password, where MFA was off	Okta advisory
Rebuy Engine (Oct 14, 2026, self-reported)	An email address on the company's own domain, never verified	Staff-level reads on internal stores and settings reads on 11 merchant stores	Co-founder and COO, LinkedIn, 22 Oct 2026

The rule all three break is the same: **an email address is a contact detail, not an identity.** Identify a user by the identity provider's immutable subject — the OpenID Connect `sub` with its issuer, in Entra the object ID with the tenant ID — require a verified email before linking any account, restrict which tenants may sign in, and grant elevated access only to a named group. MFA is necessary and, as nOAuth shows, not sufficient.

Why `/auth/me` claims and extras refuse the impostor

In the CRM Sync model, nothing an app does is decided from the email address. Every request is resolved through `/auth/me`, which reads five things in a

fixed order, and each one is a row someone had to be granted rather than a fact inferred from what the caller typed:

STEP	WHAT `/AUTH/ME` READS	WHAT IT PROVES	WHAT IT DOES NOT ACCEPT
1 · Token	A server-signed session token, checked against a logout denylist on every read	Which subject this is — the token's subject ID, nothing more	An ID, email or role sent in the request body
2 · Account status	The subject's row, before any write runs	The account is live, not deleted or tombstoned	A valid token for a deleted account
3 · **Claims** (`user_claims`)	Identity links keyed by the provider's immutable subject — Google's `sub`, the Shopify customer ID — and `email_verified` as the provider asserted it, one row per subject	Which external identities belong to this subject	An email match as proof that two logins are the same person
4 · **Extras** (`user_extras`)	Profile and preference fields, cleared when marketing consent is withdrawn	What personal data exists, and that it is consented	Profile data held by the app that renders the page
5 · Entitlement	Permissions granted to the subject, scoped to a shop	What this subject may do, and where	Privilege inferred from an email domain, a staff flag or an account's age

Platform administration sits outside this path entirely: a separate root credential and revocable tenant-scoped tokens, never a user account with an elevated flag.

INCIDENT	WHAT THE ATTACKER HAD	WHERE `/AUTH/ME` STOPS IT
Rebuy	A new, unverified account on the company's own email domain	Step 5. A new subject has no entitlement rows, and no rule anywhere grants permissions by email domain — so the account authenticates and can reach nothing. Step 3 records it as unverified, and no claim links it to any staff subject
Entra nOAuth	An attacker's tenant asserting the victim's email address	Step 3. Subjects are matched on the provider's immutable subject, not the email claim, so a different tenant's identity is a different subject with no claims, extras or entitlement of the victim's
Okta delegated authentication	A username and a cached earlier sign-in	Step 1. Identity is only ever a server-signed token for a subject ID, checked against revocation on every read; there is no username-plus-cache path to a session

Closed on 22 October 2026. Writing this section surfaced the same mistake in CRM Sync's own code, and it is fixed in production. An email-only match now attaches a login to an existing subject only when the inbox is proven — the provider verified the address, a Shopify one-time-code login, or an invitation link — and an unverified login is refused rather than attached. A typed email on a public form writes nothing until a single-use link mailed to that address is clicked, and every unconfirmed submission gets the same reply, so the form cannot reveal whether an address has an account. Rewards enrolment takes identity only from a session. Four static checks in the release gate fail if any of this regresses. The full claims-and-extras model is in Entitlement Security → claims and extras.

Merchant checks: find these apps among those installed, delisted ones included; review app embeds and custom JavaScript fields for scripts nobody added; run a test checkout in a private window and confirm it lands on Shopify's checkout domain; read Shopify's notice for your store's exposure window. Shoppers who paid by Shop Pay or PayPal were generally not exposed; shoppers who typed a card number during a window were.

Remove every client-side redirect to another domain

The hijack worked because a script in the shopper's browser could send them somewhere else. Any script that can do that today is a hijack waiting for a compromised vendor — legitimate ones included: market redirectors, affiliate rewriters, "faster checkout" buttons, leftovers from a previous storefront.

REMOVE — CLIENT SIDE	REPLACE WITH — SERVER SIDE
`window.location` or `location.href` in theme code, app embeds or tag managers	Redirect rules at the Cloudflare edge or a Worker: a 301 to an allow-listed destination
`<meta http-equiv="refresh">` pointing off-domain	The same edge redirect, or a link the shopper chooses
Scripts that rewrite links or the checkout button	Checkout posts only to the store's own Shopify checkout
Geo or market redirectors in the browser	Market routing at the edge, on Shopify Markets domains or subfolders
Old domains forwarded by script	Domain-level 301s in one redirect table under version control

Then hold it: a destination allow list for every server redirect; a Content Security Policy with `script-src` limited to an inventoried list and `form-action` limited to the store's own origins and checkout; a payment-page script inventory with change alerts (PCI DSS 6.4.3, 11.6.1); and a private-window test checkout on every release.

What is being replaced

RISK ON A MAMP/WAMP-STYLE SHARED VM	WHAT HAPPENS	WHAT REPLACES IT
One OS user for every site	A file-read or upload bug in one plugin reads every site's `wp-config.php` or `env.php`	One container per site, read-only filesystem, no shared volumes
Public origin IP	Attackers skip the CDN and WAF	Origin reachable only through a Cloudflare Tunnel
TLS ends on the box, or not at all	Plain HTTP between CDN and origin	Edge certificate, Full (strict), origin certificate or tunnel
One database user	Every app can read every table	A role per service, plus Postgres row-level security
Admin panels on the open web	`/wp-admin`, Magento admin and phpMyAdmin face everyone	Cloudflare Access on every admin path
Frozen PHP versions	End-of-life PHP survives because upgrading breaks a neighbour	A pinned image per site, rebuilt and scanned each release
Secrets in files	Keys inside the web root	Secrets injected at runtime
No audit trail	FTP edits leave no record	Git commit, image digest and deploy tag per change

The options, and what each one isolates

OPTION	TLS AND ORIGIN	CODE ISOLATION	ADMIN LOGIN	DATA ISOLATION	SCALING
Shared VM (today)	TLS on the box or none; public IP	None	Passwords on the open web	One database user	Up only
WordPress or Magento in Docker behind Cloudflare	Edge TLS, Full (strict); tunnel; no inbound ports	Container per app	Cloudflare Access	Role per service	One VM, up
EmDash on Cloudflare Workers	Edge TLS; no origin	Isolate per plugin, permissions manifest	Passkeys; SSO	Plugins have no direct database access	Pay per use
11ty + Decap CMS on Cloudflare	Edge TLS; no origin	No code runs per request	Access on `/admin` + GitHub OAuth	Static files; live data through APIs	Static requests free and unlimited
Jekyll on GitHub Pages	GitHub-managed TLS	No code runs per request	GitHub accounts	Static files	Free within 1 GB and a 100 GB/month soft limit; **not permitted for e-commerce**
Shopify Storefront Web Components on Webflow, WordPress, Astro/EmDash, 11ty or AEM	The site's own TLS edge; checkout on Shopify	No commerce code on the site — Shopify's script renders products and cart	The site's own; customer accounts on Shopify	The site stores no customers, orders or cards	Scales with the site; commerce scales on Shopify

OPTION	TLS AND ORIGIN	CODE ISOLATION	ADMIN LOGIN	DATA ISOLATION	SCALING
Shopify Functions in Rust	Runs inside Shopify	WebAssembly sandbox, declared input only	Shopify app permissions	Scoped to the shop	Shopify scales it
Kubernetes + Postgres RLS	Edge TLS; NetworkPolicy; TLS to the database	Pod and namespace	Access + SSO	Row-level security per tenant and actor	Horizontal; highest cost

Order of moves: static first, for every page that needs no login. EmDash next, where editors need a live CMS. WordPress or Magento in Docker last, only for what cannot yet leave PHP.

EmDash was announced by Cloudflare on April 1, 2026 as a v0.1.0 developer preview — MIT licensed, TypeScript on Astro, no WordPress code. Content imports from a WXR export or the EmDash Exporter plugin; themes must be ported. It has no commerce, so WooCommerce stays in Docker or the store moves to Shopify. Pilot it on a blog subdomain first. Cloudflare: EmDash

The static path has run in production for years — Jekyll and GitHub Pages since 2008, 11ty since 2017, Decap since 2016 as Netlify CMS. GitHub Pages' terms forbid running an online business or e-commerce site on it, so a store built with Jekyll or 11ty is hosted on Cloudflare.

The safer customer option: let Shopify hold the customer and the card

For a store that wants to keep its WordPress or Webflow site, the most effective PII and PCI reduction is to stop holding the data at all. Shopify's Storefront Web Components put products, prices and a cart on any page with a few lines of HTML, and the **Buy now** button sends the shopper to checkout on Shopify.

Shopify's own WordPress plugin — built by Shopify, not by WordPress — does the same from the WordPress Editor: products are managed in Shopify and placed on WordPress pages and posts.

The components are plain HTML elements plus one script, so they are not tied to a CMS. The same block works in a Webflow embed, a WordPress theme or block, an Astro or EmDash component, an 11ty template, and an AEM component or Edge Delivery block — anything that renders HTML and can load a script. Try them in Shopify's playground.

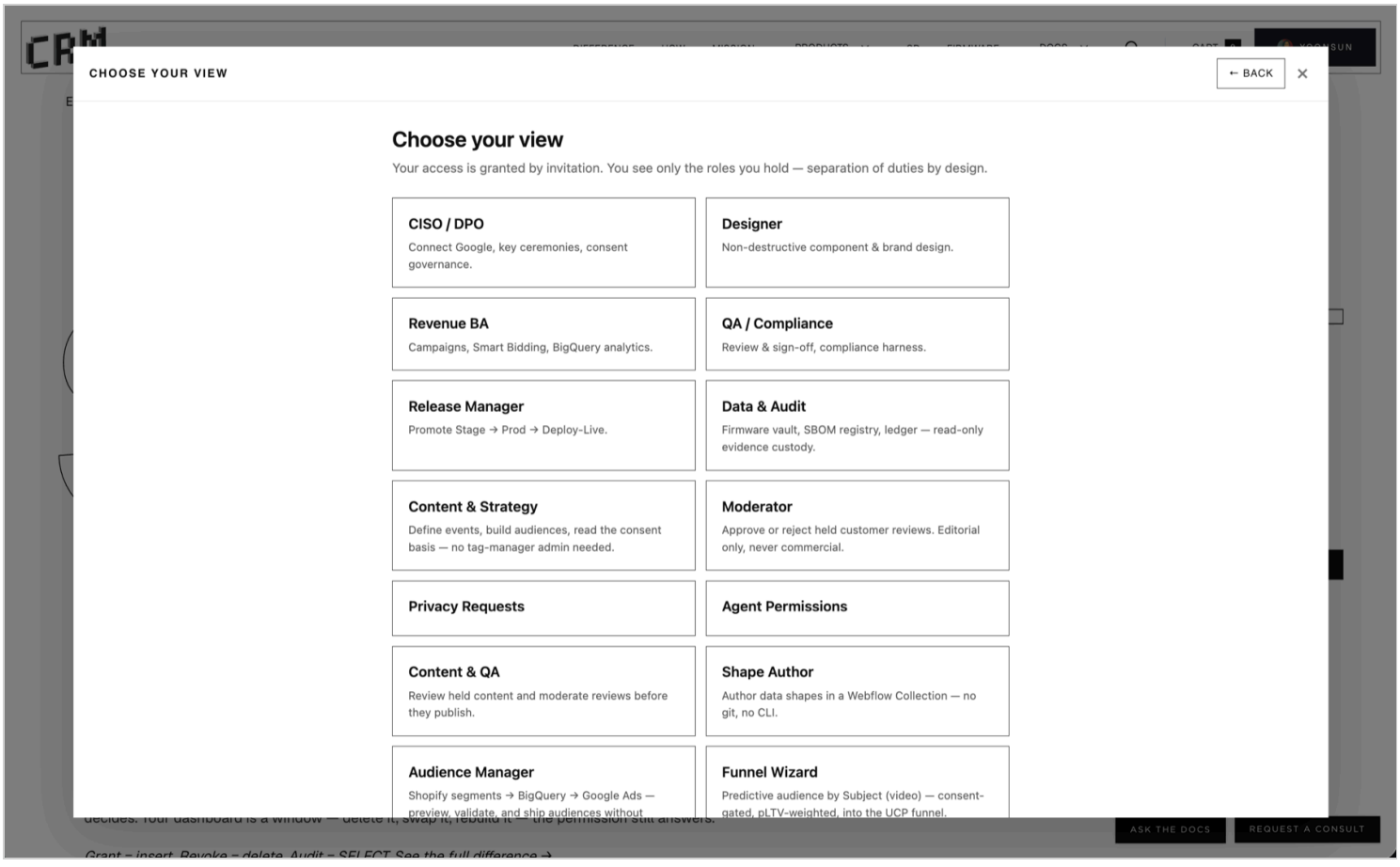
A working example. crm-base.webflow.io is a Webflow site with no commerce code of its own. When a shopper adds a product to the cart, a CRM Sync embed mounts Shopify's `<shopify-store>` and `<shopify-cart>`, and checkout lands on the Shopify store at www.crm-sync.dev. The Webflow site never sees a card number or keeps a customer record; the Shopify store holds both.

The same theme renders the same everywhere. The design is the Webflow export plus one embed block, with no framework code in it, so the same theme renders the same on WordPress, AEM, 11ty, Shopify, Next, Astro, Nuxt, Svelte and Drupal — with the forms and the data-binding security intact. The security does not live in the theme, so it does not change when the host does: forms still post to the Worker and hold a typed email until the inbox confirms, `/auth/me` still decides who the caller is, permissions are still granted rows checked on every request, and the cart and checkout are still Shopify's. None of those hosts ever holds a customer record, a consent record or a card.

The structured data carries over too. The JSON-LD for collections and their joins, the `llms.txt` answer engines read, and the LLM weighting are generated by the Worker from the records, not by the theme. Each AEO taxonomy item carries a weight — the summed predicted value of that channel's consented members, an aggregate and never a per-person score — and the JSON-LD and agent surfaces rank by it. Move the theme to another host and what an answer engine reads, and how it is ranked, stays the same.

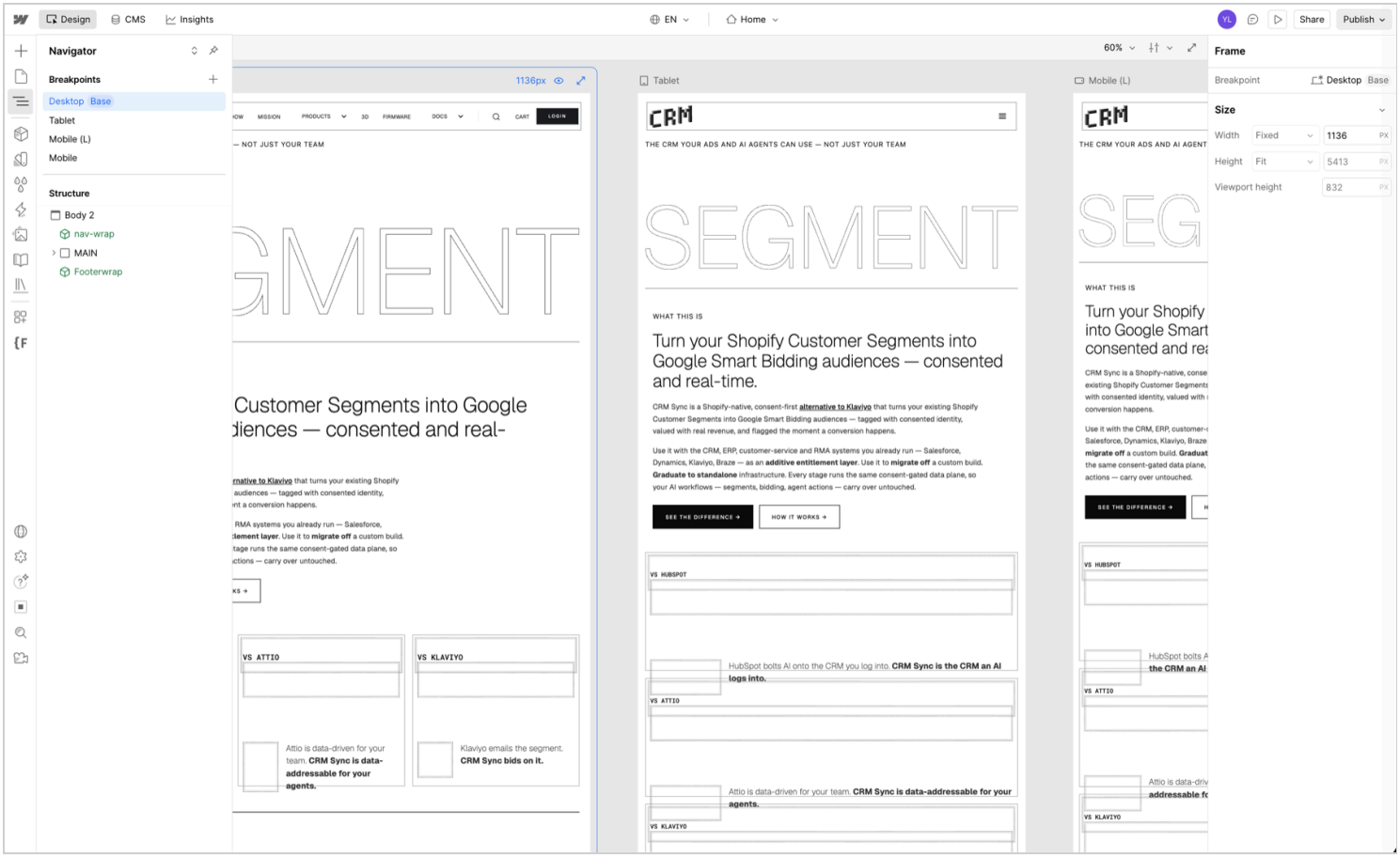
Stakeholder teams review the model

The modeling is intentional, and it is reviewable. Because the collections, joins, weights and permissions are records rather than theme code, the people accountable for them can read and adjust them without a developer redeploying a site: subject-matter experts and QA review the **content**, developers own the **schema and joins** behind the release gate, and revenue stakeholders adjust the **context** — the taxonomy, channels and weights that decide what is ranked. **Mandate permissions** are adjusted the same way: an agent's mandate can be tested and revoked from the console, and the roles are separate by design — the role that tests a mandate before a release depends on it cannot revoke it. Each change is a recorded row, so every adjustment can be checked after the fact.



Signed in as the author on www.crm-sync.dev. The name in the navigation comes from `/auth/me` *— the claims and extras for that subject, read on the server, not stored by the theme — and the views listed are exactly the roles that*

subject was granted. Access is by invitation; someone holding only Revenue BA sees campaigns, Smart Bidding and BigQuery analytics and nothing else. QA / Compliance reviews and signs off, Content & Strategy defines events and audiences, Agent Permissions holds the mandates, and Shape Author edits the data shapes in a Webflow collection with no git and no CLI. The WordPress and Astro/EmDash paths are in File System Agnostic Publishing, and the AEM path in Shopify / Google Integration on an AEM Scaffold.



The crm-base design in Webflow at three breakpoints. The structure is a navigation component, the page content and a footer component — design only. The CART link in the navigation opens Shopify's cart; the products, cart and checkout come from Shopify, not from Webflow.

OPTION	WHAT THE SITE HOLDS	WHAT SHOPIFY HOLDS	WHY IT IS SAFER
Storefront Web Components on Webflow, WordPress, Astro/EmDash, 11ty or AEM	Markup, the store domain, and optionally a public, read-only Storefront token (needed only for inventory counts and custom data)	Catalog, prices, cart, checkout, customer accounts, payments	No customer table, no order table, no card flow and no secret key on the site; a breach of the site has no customer database to take
Shopify's WordPress plugin	WordPress pages and posts that embed Shopify products	Products and checkout; Shopify states PCI, SOC and GDPR compliance are built in	WooCommerce's customer and order tables never exist in the WordPress database
WooCommerce or Magento on the site's own server (for contrast)	Customers, orders, addresses, often payment-flow scripts	Nothing	Every row of PII sits in the same PHP application as every plugin

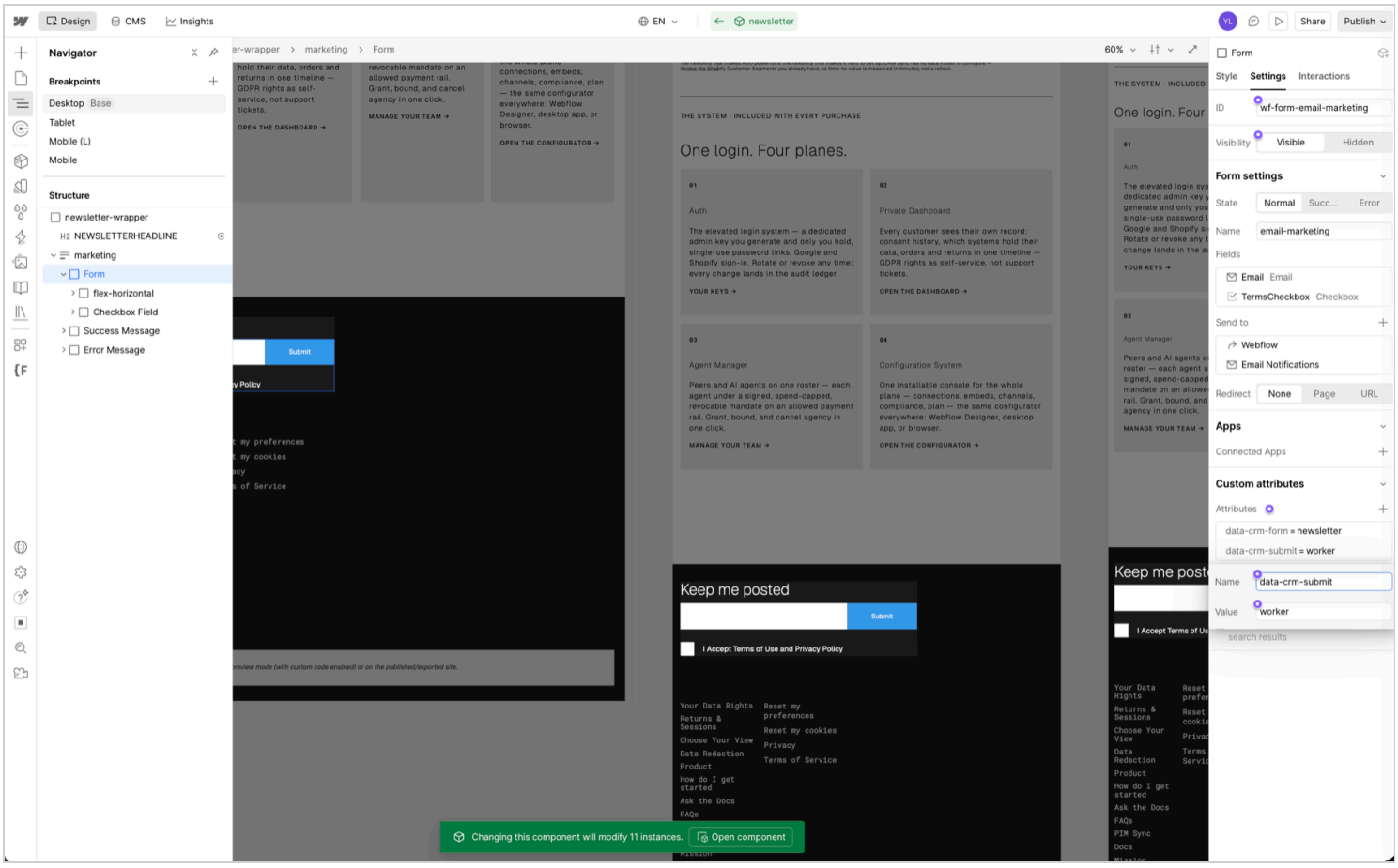
What it does not remove. The page that holds the Buy now button is still a page you serve, and the October 2026 hijack worked by changing what happened *before* checkout. The rules above still apply to it: no client-side redirects to other domains, a Content Security Policy that lists every script, an inventory of the scripts on any page that links to checkout, and a private-window test checkout on every release. WordPress still runs PHP for the content site, so the runtime patches still matter. Only the public Storefront token belongs in the page — never an Admin API token. Shopify's plugin page does not say where customer and order records live; confirm that before describing the plugin to customers as keeping them off WordPress.

Setup, including the Webflow-to-WordPress and EmDash/Astro paths, is in File System Agnostic Publishing → the Web Components block.

Secure forms on any front end

A form is the other place PII enters, and the platform's own form storage — Webflow submissions, a WordPress form plugin's table, a Liquid contact form — puts that PII next to everything else the platform runs. CRM Sync forms send the data to the Worker instead, and the form itself is only HTML:

```
<form data-crm-form="newsletter" data-crm-submit="worker">
  <input type="email" name="email" required>
  <label><input type="checkbox" required> I accept the Terms of Use and Privacy Policy</label>
  <button type="submit">Submit</button>
</form>
<script src="https://crm-sync.dev/embed/forms.js" defer></script>
```



The crm-base newsletter form in Webflow. The design stays in the Designer; the two custom attributes hand the submission to the Worker.

Because the contract is two attributes and one script, the same form works wherever HTML renders:

FRONT END	WHERE THE FORM LIVES
Webflow	A form element with the two custom attributes, as above
Astro / EmDash	A component that renders the `<form>`; the script in the layout
Next.js	A component that renders the `<form>`; the script through `next/script`
11ty	An include or shortcode; the script in the base layout
Shopify	A theme section or snippet in Liquid; the script in `theme.liquid`
WooCommerce / WordPress	A block or template part; the script enqueued by the theme
AEM	A component or Edge Delivery block that emits the `<form>`

What makes it secure.

- **The platform never stores it.** The script intercepts the submit before the platform's own handler runs and posts to the Worker, so the address is not written to Webflow, WordPress or Shopify form storage.
- **A typed email proves nothing.** Without a session, the Worker records nothing and mails a single-use, 48-hour confirmation link; the permission is written — with its date, time and the exact wording shown — only when the inbox answers. A signed-in shopper's session proves the address, so theirs is recorded immediately.
- **The reply never reveals an account.** Every unconfirmed submission gets the same answer, whether or not the address already has an account.
- **Abuse is capped.** 60 submissions an hour per IP, three confirmation mails a day per address.

The demo, on our own data. This is the live run from 22 October 2026, submitted from no session with the author's own address (masked here):

```
POST https://crm-sync.dev/forms/newsletter

{"email":"y••••••••••@gmail.com","page":"/verify-email-is-not-identity","locale":"en"}
```

```
{"ok":true,"form":"newsletter","form_id":"newsletter_en_2026-10-22T12:02:22.862Z_f767c744",

 "locale":"en","status":"confirmation_sent",

 "why":"check your inbox — the request is recorded once the address confirms it"}
```

No user ID, no "created" flag, and no consent row until the link in the inbox is clicked. The `form_id` is the join key: the same value is written to the reconcile log, the GA4 signal event and the response, and it survives confirmation, so the held entry and the recorded one join on it.

Email-only plugin forms and fee-based middleware. Most CRM and plugin forms capture an email address and nothing else — no consent wording, no time, no session — and store it in the vendor's own database, often a PHP application outside the merchant's control. Fee-based middleware then copies it onward, charging per task or per record. Moving that audience to Google is not a file transfer: Google's EEA consent rules require `ad_user_data` and `ad_personalization` consent for the people in it, and an address with no consent event behind it has nothing to show. Migrate with the consent, not just the address — export what consent evidence exists, send the same single-use confirmation to everyone whose record has none, and connect the confirmed audience to Google's APIs directly from the Worker, which also ends the per-record middleware fee.

What it does not cover. If the script is blocked — an ad blocker, a strict CSP that omits it — the platform's own handler takes the submit, and the address lands in the platform's form storage after all. Remove the platform's own destination

where the platform allows it, list `crm-sync.dev` in the CSP, and test the form with the script blocked.

Packaging the form. The HTML contract above is what ships today. Because the form is a contract — a slug, its fields, its consent wording — it can travel in other shapes:

SHAPE	WHAT IT CARRIES	STATUS
HTML + `forms.js`	The form and the submit interception	Built
Worker route `/forms/<type>`	Validation, the hold, the confirmation mail, the consent record	Built
`form_id` join key	One identifier across the reconcile log, GA4 event and consent record	Built
A Rust crate	The form's field schema and validation as a type, compiled to WebAssembly and checked inside the Worker — the same shape check used for AI task binding	Pattern, not built
Joinable data for 11ty, Liquid or Markdown	The form definition as data — `_data/forms.json` in 11ty, a Markdown front-matter block, or a Liquid snippet that reads it — so every front end renders the same fields and wording from one record	Pattern, not built
A Shopify metaobject	The same definition as a metaobject entry, so the theme renders the form from data the store already governs	Pattern, not built

Shopify plugins become Rust Shopify Functions

OLD PLUGIN PATTERN	REPLACE WITH	WHY
Line-item Script — tiers, BOGO, loyalty redemption	Discount Function	Runs inside Shopify on declared input; no merchant server to breach
Shipping Script	Delivery customization Function	Same
Payment Script	Payment customization Function	Same
Upsell or bundle app editing the cart	Cart transform Function + theme app extension	No third-party server holding order scopes
checkout.liquid fields and validation	Cart and checkout validation Function + Checkout UI extension	No arbitrary script on the payment page
Thank-you page script tag for points or pixels	HMAC-verified order webhook to a ledger Worker, plus a consent-aware web pixel	Events verified; tracking honours consent
PHP app server on the REST Admin API	GraphQL Admin API from a Worker	Meets the GraphQL-only rule; no PHP host

Shopify recommends Rust for public apps, large carts and complex logic, because it compiles directly to WebAssembly and stays inside the instruction-count limits; JavaScript is for prototypes. Functions decide; they do not store. Points, history and consent stay in the ledger, behind `/auth/me`.

Identity, data and AI tasks

1. **Identity.** A sidecar resolves every request through Xano `/auth/me` with the caller's JWE token and forwards the actor in a signed internal header. PHP never parses tokens and never trusts a `user_id` sent in a form.

2. **Data.** Postgres row-level security filters every read and write by tenant and actor, so a plugin bug or SQL injection returns only the caller's own rows. MySQL has no RLS; WooCommerce and Magento data either moves or sits behind filtered views.
3. **AI tasks.** Each automated job runs in a Tool Runner loop with a scoped tenant key and only the tools that task needs — the runner exists in PHP too. The full model — Cloudflare binds, Xano grants, Rust checks — is in Entitlement Security → AI task binding.

AI data binding with provisional permissions. A task binding says which tools an agent may call. The data it may touch is bound the same way — by permissions that expire, and by gates on where the subject is, who they share with and what device they used:

GATE	WHAT IT CHECKS	STATUS
Provisional access	Every grant carries a start and end date, and each permission can carry its own expiry; both are checked on every permission decision, so a time-limited grant ends without anyone revoking it	Built
Sovereign nation	A per-country policy, keyed by ISO code, read before an order exists: sovereign markets require a named permission (Korea's domestic settlement, for example), sanctioned or blocked markets are refused, and an unreachable policy fails closed	Built
Peer	Whether one subject may share with another	Partial — the check is written and tested but no route calls it yet; sharing is governed by single-use, expiring invitations today
Device	The platform, device type and browser behind each consent event	Recorded as provenance on every consent row; refusing access by device is not built

HOP	PERMISSIONS	TLS TRANSPORT
Browser or agent → edge	Access policy, or the Xano JWE	TLS 1.2 minimum; HSTS
Edge → origin	Origin accepts only Cloudflare	Tunnel; no inbound ports
Runner → tools	Scoped tenant key, per-task tool list	HTTPS; Access service tokens
Worker → Xano	Workspace-scoped key; permission check in Xano	HTTPS; a shared-secret header so the direct Xano URL refuses traffic that skipped the Worker
Shopify → store logic	Function input query; webhook HMAC, constant-time compare	Shopify-hosted; HTTPS webhooks
Service → database	Role per service; actor and tenant set per request	`sslmode=verify-full`

Google Merchant API: rebuild loyalty, reviews and comparison in one pass

The Google feed integration is being rebuilt regardless. The Content API for Shopping shut down on 18 August 2026; Merchant API became generally available as v1 in July 2025, its v1beta versions were discontinued on February 28, 2026, new features ship only there, and conversion sources move from `content/v2.1` to `conversions/v1`. Connect loyalty, reviews and price comparison directly in the same conversion, rather than through a plugin that can be delisted.

GOOGLE SURFACE	MERCHANT API	NEEDS	REFERENCE
Loyalty — member prices, points, loyalty shipping	`loyaltyPrograms[]` on product attributes	A program linked in Merchant Center; AU, JP, UK, DE, FR, US; member prices matching feed, landing page and checkout	Product attributes · Attribute spec
Product reviews	Reviews sub-API, `productReviews`	Product ratings enrolment; `is_verified_purchase` set from a real order	Guide
Seller reviews	Reviews sub-API, `merchantReviews`	Store ratings enrolment	Guide
Price comparison	Reports sub-API — price competitiveness, price insights, competitive visibility	Standalone or individual sub-account	Reports
Comparison Shopping Services	CSS API; `Csses` folds into Accounts	A CSS partner	CSS
Conversions	Conversions sub-API, `conversionSources`	Field renames: `property`, `destination`, `report`	Migrate

Conversions at no ad spend. Merchant Center conversion sources report conversions from free listings — the organic surfaces where reviews, loyalty member prices and price comparisons appear — as well as from ads. They only match exactly when each event carries three things, captured at the moment it happens:

CAPTURED	WHY IT MAKES THE MATCH EXACT
The GA4 session ID	Ties the conversion to the visit that produced it, not to a guess about the visitor
Device provenance	Platform, device type and browser, recorded with the consent event, so the same person on two devices stays two provable sessions
An idempotency key	Derived from the subject or session, the event, its time and its source (`form_id`), so a retry or a double-fired tag is one conversion, never two

That is what turns an AI answer into an attributable sale: the assistant finds the candidate, and the session, device and key decide the transaction. CRM Sync writes all three today for a signed-in submission. A submission confirmed by email deliberately drops the typing browser's GA identifiers — carrying them would bind a stranger's browser to the person who clicked the link — so it keeps the device and the key but not the session. Conversions sent without all three are estimates.

A verified-purchase review or a member price shown to Google is a claim about a real order and a real member. CRM Sync records that claim as a signed mandate anyone can check offline in a browser — verify a mandate · verified transactions.

Supply chain: every new ecosystem is a new door

Moving off PHP brings in Python for AI gateways, npm for Workers and Rust crates for Shopify Functions. On March 24, 2026, LiteLLM 1.82.7 and 1.82.8 were published to PyPI with a credential stealer — one through a `.pth` file that runs on every Python start — taking SSH keys, cloud and Kubernetes credentials, `.env` files and TLS private keys (BleepingComputer). A second poisoned release, 1.107.0, followed on October 21 — Entitlement Security → LiteLLM.

ECOSYSTEM	PERMISSIONS CHECK BEFORE INSTALL	WHAT IT CATCHES
Rust crates	`cargo vet --locked` in CI: every version covered by a trusted published audit, or reviewed and certified	A hijacked release nobody has reported yet
Python	Hash-pinned installs; versions at least two weeks old	A new malicious release
npm	Committed lockfile; `npm ci --ignore-scripts`; an osv-scanner baseline with expiry dates	Install-time scripts; known-bad versions
GitHub Actions	Pinned to commit SHAs	A retagged action

CRM Sync's desktop build imports the published crate audits of Mozilla, Google, the Bytecode Alliance, ISRG, Embark and Zcash — 102 crates fully audited, 357 recorded as a starting baseline — and fails CI on any crate version no audit covers. Wildcard trust is avoided on purpose: a hijacked publisher would pass it.

Signed records: SBOMs, firmware and signing keys

Every incident in this article ends with the same question from the board, the insurer and the regulator: *were we affected, and can you prove it?* A business that has to answer by meeting finds out in weeks. One that keeps signed records answers by query in minutes. Three records do the work:

- **A software bill of materials (SBOM)** is the ingredient list for each release: every package, runtime and certificate bundle, with its version. When a flaw is announced, the SBOM says which releases contain it.
- **A signed firmware or build image** is the sealed box. Its hash is certified when it is uploaded, so a distributor, a device or an auditor can check that what they received is exactly what was built.

- **A signing key** is the seal itself. Its private half stays on the server. Its public half is published, so anyone can check a receipt, a mandate or a certificate without asking the issuer.

CRM Sync publishes all three. Anyone can check a mandate offline in a browser on the **verify plane**, against the public keys at `/.well-known/jwks.json`. The app's own SBOM is public, and firmware images go to the firmware vault. The vault gives each image its own encryption, an upload certificate, grant-gated downloads, and a hash-chained ledger of every grant, denial and download.

The breaches, on one timeline

DATE	WHAT BROKE	LAYER	WHAT A SIGNED RECORD ANSWERS
30 April 2025	A key ceremony at the Indian certificate authority **eMudhra** generated eight intermediate CA keys with too little randomness. Nobody noticed for about 18 months	Signing keys	Which of our certificates chain through those intermediates
24 March 2026	**LiteLLM** 1.82.7 and 1.82.8 on PyPI carried a credential stealer, taking SSH keys, cloud credentials, `.env` files and TLS private keys	Python / AI gateway	Which builds installed those versions, and which keys that environment could read, so you know what to rotate
10–20 October 2026	The checkout hijack through trusted Shopify apps, with vendor-side configuration changed (above)	App identity	Which grants each app held, and when each one was used
17–20 October 2026	A researcher recovered all eight **eMudhra** private keys from the public certificates in under a second: six TLS intermediates and two code-signing intermediates. Chrome, Mozilla, Apple and Microsoft blocked them between 17 and 20 October. Chrome removes the emSign ECC Root CA - G3 from devices from 27 October	TLS / code signing	Which runtimes and trust bundles in our releases still accept the chain
20 October 2026	Go, NSS, Ubuntu, Debian and **Composer** — the PHP dependency manager that Drupal, Magento and Joomla projects are installed with — shipped fixes to their trust stores	PHP dependency chain	Which PHP estates have taken the updated certificate bundle
21 October 2026	**php2shell** (CVE-2026-109618 and CVE-2026-109619): unauthenticated remote code execution in the PHP runtime. This covers every WordPress, Drupal, Magento, Joomla and Laravel host, including its plugins, whatever the application code	PHP runtime	Which origins run an affected PHP build, by version, without logging into each server

DATE	WHAT BROKE	LAYER	WHAT A SIGNED RECORD ANSWERS
21 October 2026	**Node.js** CVE-2026-111497, a Brotli out-of-bounds write that allows remote code execution. The same releases — 22.23.4, 24.22.1 and 26.12.1 — also reject the eMudhra chains	Node runtime	Which containers, CI runners and self-hosted apps pin an older Node
21 October 2026	**LiteLLM** 1.107.0 on PyPI and Docker, the project's second compromise this year, again stealing signing keys and AI provider keys	Python / AI gateway	Which images resolved the floating tags, and which mandates were signed by a key that environment held

Details and sources are in Entitlement Security and in reporting on eMudhra by The Hacker News and Help Net Security.

What eMudhra teaches about signing keys

Who eMudhra is. eMudhra is an Indian digital-trust company. Its main lines of business:

- **India's largest licensed certifying authority.** Under India's IT Act it issues Digital Signature Certificates, which people and companies use to sign official filings: income tax, GST returns, company filings and government e-tenders. This is the core of the business.
- **eSign.** An online signing service built on Aadhaar, India's national digital identity.
- **emSign, its publicly trusted CA brand.** Its roots sit in the Chrome, Mozilla, Apple and Microsoft trust stores, so it can issue TLS and code-signing certificates that browsers accept worldwide. This is the part the incident hit, and one of the few Indian-owned roots in those programs.
- **Trust software for others.** Certificate-authority and authentication products for governments, banks and enterprises, in India and abroad.
- **A listed company,** public in India since 2022 and expanding into the Middle East, Africa, Southeast Asia and the US.

eMudhra says TLS is about 1% of its revenue, and its shares fell about 20% after the disclosure. Both numbers make the same point. A small, rarely run ceremony got little scrutiny, yet the failure hit the company's whole product, because what it sells is trust. India's Digital Signature Certificates run under India's own CA licensing rather than the browser root programs, so as far as has been reported they are not directly affected.

The keys were not stolen. They were computable. The ceremony used certified hardware security modules and still produced weak keys. A key is only as strong as the randomness it was made from, and an audited process is not a substitute for checking the output. Four properties keep a signing key's failure small:

PROPERTY	WHAT CRM SYNC DOES	WHAT IT LIMITS
Generated by the platform's cryptographic random source, never by hand or by a copied script	Mandate and entitlement keys are Ed25519, generated with WebCrypto inside the Worker	Weak or reused randomness
Public half published with a key id	Every token names its `kid`, and the verify plane rejects any `kid` that is not in the published JWKS	Trusting a key nobody can look up
Rotation with an end date	Keys rotate through a recorded key ceremony. A retired key keeps verifying what it already signed until its `not_after`, then stops	The useful life of a leaked key, without voiding honest receipts
Revocation you control	Retiring a CRM Sync key means editing our own JWKS. A browser vendor does not need to act	The days a public CA needs

LiteLLM is the reason signing keys never sit in an application environment. Both poisoned releases searched for signing keys and `.env` files. A key held as a platform secret and used only by the signing service is not on the disk the payload reads.

The number 4, and what row-level controls still protect

What an elliptic-curve key is. An elliptic curve is a fixed, published mathematical curve, with a fixed starting point on it, *G*. "Multiplying" a point by a number means stepping along the curve that many times by a special rule. The rule is easy to run forwards: given a number, finding the point is fast. It is effectively impossible to run backwards: given only the point, there is no shortcut to the number. That one-way step is the whole key pair:

- the **private key** is the number
- the **public key** is the point it lands on

Elliptic-curve keys are much shorter than RSA keys of the same strength, which is why certificates, phones and Ed25519 signatures use them.

eMudhra's keys were elliptic-curve keys, not RSA. An elliptic-curve private key is a single number, *d*. The public key is that number multiplied by a fixed, published starting point *G*: public key = *d* × *G*. Picked at random from roughly 2^256 possibilities, *d* cannot be worked back from the public key.

For one of eMudhra's keys, *d* was 4. Anyone could compute 1×*G*, 2×*G*, 3×*G* and 4×*G* and match the published key on the fourth try. A researcher recovered all eight keys from the public certificates in under a second, so all of them were tiny. `openssl rand -hex 32` draws *d* from the full 2^256 range. Size was never eMudhra's problem; the randomness was missing.

Why elliptic curves? The business reason. Elliptic-curve keys give the same security as RSA for far less size and work:

	ELLIPTIC CURVE (256-BIT)	RSA OF THE SAME STRENGTH (ABOUT 3,072-BIT)
Public key	32–64 bytes	About 384 bytes
Signature	64 bytes	About 384 bytes
Making a key	Milliseconds, even on a phone	Much slower — two large primes must be found
Signing	Fast	Several times slower
Fits in a QR code or a handle	Yes	No

Smaller and faster means less bandwidth, quicker page loads, longer battery life and lighter servers, on every handshake and every signature.

- **A certificate authority**, eMudhra included, builds an elliptic-curve hierarchy for the usual reasons. eMudhra has not, as far as we have seen, stated its own.
  - Customers want lighter, faster HTTPS.
  - Browsers and TLS 1.3 already favour elliptic curves.
  - A new elliptic-curve root keeps the CA in browser root programs for the next decade as RSA roots age.
  - Everything it issues and revokes gets cheaper.
- **Nostr** chose Bitcoin's own curve (secp256k1) and signatures (BIP-340).
  - Tested libraries, hardware signers and wallets already existed, and the same key sits beside Lightning payments.
  - A 32-byte public key is short enough to be the handle and to fit in a QR code.
  - Relays verify every post, so small, fast signatures keep a network nobody owns cheap to run.
  - A phone makes a key instantly, with no sign-up server.

The curve was not the mistake. Weak randomness breaks RSA just as surely:

- **Debian, 2008:** an OpenSSL bug left only about 32,000 possible keys on Debian systems.
- **"Mining your Ps and Qs", 2012:** researchers found thousands of internet devices whose RSA keys shared primes, which made them computable.

Elliptic curves are slightly less forgiving of carelessness, though. ECDSA leaks the key if a single signature's random number is weak, which is why newer systems prefer EdDSA. The next cost is shared by both: a large quantum computer would break elliptic curves and RSA alike. The post-quantum replacements (ML-DSA for signatures, ML-KEM for key exchange) have much larger keys and signatures, so the size advantage that drove these choices will shrink in the migration.

RSA, elliptic curve and EdDSA — what each one is. RSA and elliptic curve are two kinds of key. EdDSA is not a third kind: it is a way of *signing* with an elliptic-curve key.

	RSA	ELLIPTIC CURVE
The private key is	Two large secret prime numbers	One secret number
The public key is	Their product — a very large number	A point on a published curve, reached by "multiplying" the starting point by that number
Why it is hard to break	Splitting a huge number back into its primes is effectively impossible	Working back from the point to the number is effectively impossible
Size for the same strength	About 3,072 bits	256 bits
What it can do	Sign and encrypt	Sign (ECDSA, EdDSA) and agree a shared key (ECDH)
Where you see it	Older certificates, older SSH keys, many enterprise systems	Most modern HTTPS certificates, phones, Bitcoin, Nostr, eMudhra's hierarchy

SIGNING WITH AN ELLIPTIC-CURVE KEY	ECDSA	EDDSA (ED25519 IS THE COMMON FORM)
Age	Standardised around 2000	Published 2011, standardised 2017
The one-time number in each signature	Must be freshly random. Weak or reused once, and the key can be computed from two signatures	Derived from the key and the message, so a bad random source at signing cannot leak the key
Settings to get wrong	Several curves and options	Essentially one choice per curve
Speed and side channels	Good, with careful implementation	Fast, and designed to resist timing attacks
Where you see it	Most TLS certificates, eMudhra's chain included; Bitcoin before Taproot	`ssh-ed25519` keys, Tor, and CRM Sync's mandates and receipts — the `EdDSA` in the token header the verify plane checks

Nostr uses a third elliptic-curve signature, Schnorr (BIP-340). It is a close relative of EdDSA that also mixes fresh randomness into the one-time number.

In one line each:

- **RSA:** big keys built on prime numbers. Well understood and widely supported, but heavy.
- **Elliptic curve:** small keys built on a curve. The same strength, much lighter.
- **EdDSA:** a modern way to sign with an elliptic-curve key that removes the most common way to lose it.

None of them survives a key made without real randomness — eMudhra's failure, and Debian's RSA failure in 2008 — and a large quantum computer would break all three. The post-quantum replacement for signatures is ML-DSA.

Which layer broke. A certificate authority's key does not encrypt anything of yours. It *vouches* for other people's certificates. Whoever held eMudhra's keys

could issue a "valid" certificate for any domain — a Xano instance, a store, pypi.org — and every client that still trusted emSign would accept it.

	THE CA REGISTRY (EMUDHRA)	THE DATA PLANE (XANO BEHIND CLOUDFLARE)
What the key does	Vouches for other people's certificates, for any domain	Encrypts one connection. Xano's certificate secures only Xano
Who vouches for CRM Sync's hosts	Not eMudhra	The Xano instance: Let's Encrypt, chaining to ISRG Root X1, with a 90-day certificate renewed automatically. `crm-sync.dev`: Google Trust Services
Why that is not enough	An attacker does not need your CA. They present a forged certificate to any client that still trusts emSign	The exposure is the *client's* trust store: a laptop, a container, a server — not the host's choice of CA
Scope of a failure	Every site, worldwide, for every client not yet updated	One connection

What still holds if a connection is intercepted. Nothing at the row level fixes a broken CA — updating trust stores does. But row-level design decides how much an intercepted connection can give away:

- **Sealed identity.** The session token is encrypted (JWE), and only the data plane decrypts it. Someone reading the traffic cannot read or alter the claims inside. They can replay the whole token, which is why logout writes it to a denylist checked on every read, and why sessions are short.
- **Row-level authorization.** Every read and write takes the caller's identity from the decrypted token on the server — the `/auth/me` claims and extras — never from a user id the browser sends. A captured session exposes **one person's rows, not the table.**
- **Field-level encryption.** A field encrypted with a key the application holds stays ciphertext even when its row is read.
- **The exception is a master key.** An admin key or a data-plane API key that crosses an intercepted connection has no row limit. That is a layer

1 or layer 6 key in the key layers: rotate it through the ceremony, from a patched machine.

EdDSA vs. token gating — proof is not permission. Two ideas get blurred together:

	EDDSA SIGNATURE (ED25519)	TOKEN GATING
Question it answers	*Who issued this, and has it been changed?*	*Should the holder be let in?*
How	The issuer signs with a private key; anyone checks it with the published public key, offline	The server lets in whoever presents a valid token — possession is the test
What it proves	Authenticity and integrity. A changed character fails the check	Only that someone holds the token
What it does not prove	That the person presenting it is the person it was issued to	That the token was not copied, replayed or stolen
In CRM Sync	Mandates, receipts and certificates, checked on the verify plane against the JWKS	Sessions and entitlements: the server decrypts the token, checks the denylist and expiry, then applies the caller's roles row by row

A signed token used as a gate is still a **bearer** credential: whoever holds it, gets in. So signing is never the whole answer. The gate adds:

- a short expiry
- revocation checked on every read
- an audience, so a token minted for one service is refused by another
- spending caps and scopes on agent mandates
- server-side role checks on each row

Signing makes a token **trustworthy**. Gating decides what that trust is **worth**. The same split runs through web3 "token gating", where holding an NFT opens a door: the chain proves the token is genuine, not that the wallet holding it is the person it was meant for.

Why EdDSA, after eMudhra. eMudhra's chain used ECDSA, the older elliptic-curve signature scheme. ECDSA needs a fresh random number for *every signature*. Reuse one, or let it be guessable, and the private key can be computed from two signatures — a well-known failure. EdDSA derives that number from the key and the message, so a weak random source at signing time cannot leak the key.

EdDSA would not have saved eMudhra. Their failure was in *making* the key, and every scheme needs real randomness at key generation. That is why CRM Sync's Ed25519 keys come from the platform's cryptographic random source inside the Worker, never from a hand-run script.

Nostr: when the key is the identity

Nostr, the messaging protocol Jack Dorsey championed and funded, uses the same kind of key as eMudhra — elliptic curve — but makes the opposite choice: **the key is the identity and the permission at once.**

- **The account is a key pair** on secp256k1, Bitcoin's curve, with Schnorr signatures.
- **The public key (`npub…`) is the handle.** The private key (`nsec…`) is the password, the account and the signature, all at once.
- **Every post is a signed event.** Relays — the servers — check the signature and pass the event on. No server issues the identity or can take it away.
- **The optional NIP-05 name** (`name@domain`) is only a file on a website that points a name at the key.

	NOSTR	EMUDHRA (A CERTIFICATE AUTHORITY)	CRM SYNC
Who vouches for a key	Nobody — trust comes from who follows it	The CA, for any domain	The issuer's own published JWKS
Who makes the key	The user's app, on the user's device	The CA, in a ceremony	The Worker, from the platform's cryptographic random source
What a signature means	"This key wrote it" — which is also the permission to post	"This CA vouches for this site"	"We issued this and it has not changed" — proof only
Who decides access	Nobody beyond the signature; relays may add their own gates (paid relays, allowlists)	The client's trust store	The server, every time: expiry, revocation, roles per row
Rotation	None built in. A new key is a new identity, without its followers	Revoke and reissue — if clients check	Key ids, end dates and a JWKS edit; old receipts keep verifying
If the private key leaks	Anyone can post as you, permanently; there is no recovery	Fake certificates for any site, until trust stores update	Limited to that key's layer, and rotated through the ceremony
If the private key is lost	The identity is gone	—	Rotate; nothing is lost, because the server holds the records

What Nostr shows about everything above.

- **Same kind of key, opposite outcome: Nostr is randomized where eMudhra was not.** Both use elliptic-curve keys — a private key that is a single number — though not the same curve.
  - **Key creation.** Each Nostr app makes its key on the user's device, normally from the operating system's cryptographic random source, so the number is one of roughly 2^256 — not 4.

    - **Signing.** Nostr's Schnorr signatures (BIP-340) build each signature's one-time number from the key, the message and fresh random bytes. A weak random source at signing time therefore cannot leak the key — the ECDSA failure that eMudhra's chain is exposed to.
    - **The limits.** The protocol cannot check how a key was made. There is no ceremony and no authority, so a badly written app could still produce a weak key, and nothing in the network would notice.
    - **Scale, not kind.** A weak Nostr key exposes one person. A weak CA key exposes every site that CA can vouch for. eMudhra's central ceremony was supposed to be the careful one, and it failed once, for everyone.

  The lesson is the same for both: the type of key was never the problem. What matters is whether the private key came from real randomness, and whether anyone checked.

- **Pasting an `nsec` into apps is the key-ceremony problem.** Every app it is pasted into — including AI tools and browser extensions — can take the identity for good. Nostr's answer is a remote signer (NIP-46, a "bunker"): the key stays in one place, and apps ask it to sign. That is the ceremony's rule — the thing that holds the key is not the thing that asks for its use.
- **Proof is not permission — unless you choose to merge them.** Nostr merges them on purpose, and gains censorship resistance and independence from any platform. The price is a leaked key that cannot be revoked and no row-level limit on what it can do. Commerce cannot pay that price: a stolen customer or admin key has to be stoppable today. So CRM Sync keeps the signature as proof and leaves the decision to a server that can say no.
- **eMudhra touches Nostr only at the edges.** Post signatures do not depend on certificate authorities, so a forged certificate cannot forge a post. But relays are reached over secure WebSockets and NIP-05 names are fetched over

HTTPS. An intercepted connection could fake a name or hide posts — it could not sign as you.

Details are from the Nostr specifications (NIP-01, NIP-05, NIP-19, NIP-46) — see the NIPs repository.

Non-random keys were never safe — AI only shortens the window

Would eMudhra's keys have been fine before AI? No. A private key of 4 is found by trying 1, 2, 3, 4 — a pocket calculator, not a model. The keys were breakable from the ceremony on 30 April 2025. Every well-known weak-key failure was weak the moment it was made, and humans found each one:

YEAR	INCIDENT	WHAT WAS WRONG	FOUND BY
2008	Debian OpenSSL	A code change left only about 32,000 possible keys	A developer reading the code
2010	Sony PlayStation 3	ECDSA signing reused the same "random" number every time, so the private key could be computed	Hobbyist researchers (fail0verflow)
2012	"Mining your Ps and Qs"	Thousands of devices made RSA keys with too little randomness at boot; many shared primes	Academic researchers scanning the internet
2017	ROCA (CVE-2017-15361)	A smart-card chip's library made RSA keys with a hidden structure — about 750,000 Estonian ID cards among those affected	Academic researchers
2025–26	eMudhra	Eight CA keys, one of them the number 4	An independent researcher checking published CA certificates

What AI changes is the window, not the maths.

- **Searching is cheap now.** Scanning every certificate in the public CA database or the Certificate Transparency logs for weak keys was always possible. It is now cheap, quick and automated, for researchers and attackers alike. AI-assisted tools are already credited with finding flaws such as php2shell.

- **Mistakes no longer sit quietly.** eMudhra's keys sat for about 18 months before anyone looked. Today, expect a mistake like that to be found in days, possibly by someone who will not report it.

So checking has to happen when a key is made, not years later:

- generate from a cryptographic random source
- lint the certificates
- test new keys against known-weak patterns
- monitor Certificate Transparency for your domains
- record key fingerprints, so a duplicate or reused key stands out

Security levels, in one table. Strength is counted in *bits of security*: 128 bits means an attacker needs about 2^128 tries, far beyond any computer that exists. Different key types reach that level at different sizes.

SYSTEM	KEY	SECURITY LEVEL AS DESIGNED	SECURITY LEVEL AS ACTUALLY MADE
eMudhra intermediates	Elliptic curve (ECDSA)	About 128 bits or more	**Effectively none** — all eight recovered in under a second
Nostr accounts	secp256k1, Schnorr (BIP-340)	About 128 bits	About 128 bits **if the app used a proper random source**; nothing in the protocol checks
CRM Sync mandates and receipts	Ed25519 (EdDSA)	About 128 bits	About 128 bits — generated by WebCrypto in the Worker
CRM Sync agent-commerce signing (UCP)	ECDSA P-256	About 128 bits	About 128 bits — generated by WebCrypto; ECDSA because the standard requires it
Google service-account keys	RSA-2048	About 112 bits	About 112 bits — issued by Google
CRM Sync vault and config encryption	AES-256-GCM	256 bits (about 128 against a future quantum computer)	Depends on the master secret it is derived from — see the key layers

Nostr's security level, plainly.

- **On paper it matches CRM Sync's signing:** about 128 bits, on a curve that has protected Bitcoin for years.
- **In practice, the weak points are operational, not mathematical:**
  - each app must use a proper random source, and nothing in the protocol checks
  - there is no rotation or revocation, so one leaked `nsec` is permanent
  - pasting the key into apps exposes it — the remote signer (NIP-46) exists for that reason
- **Message privacy depends on which version an app uses.** Nostr's older direct-message encryption (NIP-04) is deprecated for known weaknesses and replaced by NIP-44. Apps still on the old scheme protect less than the signatures suggest.
- **The shared limit:** a large quantum computer would break Nostr's keys, eMudhra's and CRM Sync's signing keys alike.

The level a key *could* have is set by its type. The level it *does* have is set by how it was made and how it is kept. eMudhra had the right type and none of the second.

One architecture under any front end, whatever the platform plan

The controls in this article do not belong to WordPress, Shopify, AEM or a static site generator. They sit in two layers the front end calls:

- **Cloudflare at the edge:** certificates, the WAF, Access, and Workers for forms, sign-in, signing and consent.
- **Xano as the data plane:** identity, rows, entitlements and encryption.

The front end renders pages and calls the edge. It never holds a key and never stores a customer row.

```
Browser ──TLS──► Front end (WordPress · Shopify theme · AEM · static 11ty/Astro/Jekyll)
                    │  renders pages; holds no secrets, stores no customer rows
                    ▼
                Cloudflare edge ── certificates (auto-renewed) · WAF · Access · rate limits
                    │  Worker: forms held until confirmed · /auth/me · signing · consent
                    ▼  (secrets live here, set by the key ceremony)
                Xano data plane ── sealed identity · row-level authorization · field encryption
                                · entitlements · hash-chained records
```

FRONT END	STAYS ON THE PLATFORM	MOVES TO CLOUDFLARE + XANO	HOW IT CONNECTS
WordPress	Pages, posts, the theme	Forms, sign-in, customer records, keys. Card checkout goes to Shopify through the Shopify plugin	One script tag and two form attributes; the PHP origin sits behind Cloudflare
Shopify	Catalog, cart, checkout, payments — the PCI scope stays with Shopify	Consent, loyalty and review records, agent mandates, signed receipts	Theme app embed or Storefront Web Components
AEM	Content, components, the DAM	The same as WordPress	A client library or embed in the page component
Static (11ty, Astro, Jekyll, Webflow export)	Built HTML on any host	Everything dynamic	The same script and attributes; nothing to patch on the host

Why the plan or fee does not matter. Shopify Basic or Plus, WordPress.com or self-hosted, an AEM licence or a free GitHub Pages site — the protection is the same, because none of it runs on the front end's plan:

- the key ceremony and key layers apply to secrets the front end never sees
- forms are held until the inbox confirms, at the edge
- identity is sealed and rows are authorized in the data plane

- certificates renew at the edge

A platform upgrade buys features, not these controls. Changing platform does not lose them either. The front end can be replaced, and the same edge and data plane carry over unchanged, along with their records.

What still depends on the front end:

- **Its own runtime must be patched.** php2shell on a WordPress origin is still the host's to fix, and the edge only buys time.
- **Its theme must not leak.** No client-side redirects to other domains, and no keys in theme code.

The working example — one theme, one form, one sign-in across platforms — is in the safer customer option and secure forms on any front end.

The key ceremony: a person makes the key, the AI never sees it

eMudhra shows what goes wrong when keys are made badly. LiteLLM shows where good keys leak: terminal output, shell history, the clipboard, `.env` files and chat transcripts. Today those are also exactly what an AI coding assistant reads. A **key ceremony** is the routine that keeps a secret out of all of them.

The rule is simple: **a person runs the step that creates the secret, and the AI only prepares, checks and records.** The secret goes from the generator straight into the secrets store, through a pipe or a prompt. It is never printed, never pasted into a chat, and never written into a command line.

WHO	DOES	NEVER SEES
The key holder (a security role, not the developer who asked for the change)	Runs the command that creates and stores the secret, in their own terminal	—
The AI assistant	Writes the steps, checks the result by fingerprint and behaviour, files the record	The secret
The record	Who, which secret, when, its fingerprint, how it was checked	The secret

Directions — OpenSSL and Wrangler, interactive. Run these from the Worker's folder.

Before you start: patch the machine the ceremony runs on. `wrangler secret put` sends the new key to Cloudflare over HTTPS, and Wrangler runs on Node. If that machine's Node still trusts the eMudhra chains, a fresh key could travel over an intercepted connection. Use Node 22.23.4, 24.22.1 or 26.12.1 or later. If Python is on the machine, follow Python and Docker after eMudhra first.

1. **Open your own terminal window, not the AI's.** If you work in an AI coding tool, do not run ceremony commands through it. That includes shortcuts that run a command inside the session, because their output becomes part of the conversation.
2. **Generate and store in one step, so the value is never displayed:**

   ```
   openssl rand -hex 32 | npx wrangler secret put ADMIN_KEY --config wrangler.toml --env=""
   ```

   Wrangler reads the value from the pipe. Nothing is echoed, and the shell history holds the command but no secret.

   How big is it? `rand -hex 32` means 32 random bytes: 256 bits, written as 64 hexadecimal characters (0–9, a–f). That is 2^256 possible keys, a number 78

digits long. Guessing at a trillion tries a second would take far longer than the age of the universe.

Why it is safer than eMudhra's keys. eMudhra's keys were not short. They were made badly. A wrap-and-unwrap step in the ceremony produced keys with so little real randomness that they could be worked out from the public certificates in under a second. Length is only worth anything if every bit is random. `openssl rand` takes its bytes straight from the operating system's cryptographic random source, with no extra step between the source and the key. The scale is different too. This secret guards one Worker, nobody else trusts it, and replacing it is one command. A CA key is trusted by every browser, and replacing it took four browser vendors and a root removal.

3. **For a secret someone else issued** (an OAuth client secret, a payment-provider key), use the interactive prompt:

```
npx wrangler secret put GOOGLE_CLIENT_SECRET --config wrangler.toml --env=""
```

Run the command **first**. Then copy the secret from the provider's dashboard and paste it at the "Enter a secret value" prompt. Done in the other order, copying the command overwrites the secret on the clipboard, and the command text is stored as the secret.

4. **When one secret must go to two systems** — say a Xano environment variable and the Worker — write it to a file only you can read, and feed both from that file:

```
( umask 077; openssl rand -hex 32 | tr -d '\n' > ~/.crm-ingest-key )
npx wrangler secret put INGEST_KEY --config wrangler.toml --env="" < ~/.crm-ingest-key
pbcopy < ~/.crm-ingest-key     # paste into Xano, then clear the clipboard:
pbcopy < /dev/null
```

5. **Check by fingerprint, never by value.** The AI can be shown a fingerprint: the first eight characters of a hash, which say nothing usable about a 256-bit key.

   ```
   shasum -a 256 ~/.crm-ingest-key | cut -c1-8

   npx wrangler secret list --config wrangler.toml --env=""   # names only, never values
   ```

   Then check behaviour. The old key is refused, and the new key works. Any check refers to the file (`$(cat ~/.crm-ingest-key)`) and never repeats the value, so the AI's command and output both stay clean.

6. **Record it, then delete the file.** File the record: who ran it, which secret, when, the fingerprint and the check result. Then delete any file you made.

Signing keys are not made this way at all. The Ed25519 keys behind mandates and receipts are generated by WebCrypto inside the Worker. Rotation is a server action. The private half never exists on a laptop, and only the public half is ever published. OpenSSL and Wrangler are for **shared secrets**: admin keys, webhook secrets, API keys from other providers.

You manage the keys you hold — create, rotate, revoke — and the rest of the plane from one console: Configuration System. Every change lands in the audit ledger. The full ceremony, with roles and the record format, is in Rotate a Key in Three Steps and the Key Management Lifecycle.

Dev, Stage and Prod: a separate ceremony for each, least privilege everywhere

A key that works in two environments is one leak away from both. Each environment gets **its own keys, made in its own ceremony**, and no value is ever copied from one to another. The fingerprints prove it: if Stage and Prod show the same eight characters, something was copied.

ENVIRONMENT	WHERE IT RUNS	KEYS	WHO CAN SET THEM	COMMAND
Dev	The developer's machine (`wrangler dev`)	Throwaway values in an uncommitted `.dev.vars` ; no real customer or provider keys	The developer	`openssl rand -hex 16` into `.dev.vars`
Stage	A separate Worker, `[env.staging]` , with its own storage	Generated for Stage only; test-mode provider keys	The release engineer	`openssl rand -hex 32 | npx wrangler secret put NAME --config wrangler.toml --env=staging`
Prod	The production Worker	Generated for Prod only; live provider keys	The key holder, from an approved machine on the company network	`openssl rand -hex 32 | npx wrangler secret put NAME --config wrangler.toml --env=""`

Passing `--env` explicitly matters. With more than one environment defined, a bare Wrangler command falls back to the top level, which is right only by accident.

Least privilege: each person, machine and test gets the smallest key that does its job.

- **Production secrets are set on-premises only.** The Cloudflare API token that can write Prod secrets belongs to the key holder. It is scoped to the one account and the Worker permissions it needs, and it can be locked to the company network's addresses with Cloudflare's client-IP filter on API tokens. Admin paths sit behind Cloudflare Access with a device check. A token copied off the network does not work.
- **Supply-chain tests never see a secret.** Dependency scans, crate audits and the static test harness run on every pull request, so they treat every change

as untrusted. CRM Sync's pipeline shows the pattern:

  - the job token can only read the repository
  - actions are pinned to commit hashes, not movable tags
  - installs run with `--ignore-scripts`
  - the contract tests touch only public and fail-closed paths, so they are safe on pull requests from outside forks

  The one credential the test job holds is a single-repository, read-only token for a private test dependency.

- **Deploy is a separate job with separate rules.** It runs only on a merge to `main`, never on a pull request, and it goes through the same deploy guard and test gate as a deploy from a laptop. Worker secrets are not stored in CI at all. They are set by the ceremony, so a poisoned dependency in CI has nothing to steal.
- **Reviewers approve; they do not hold keys.** Design, content and compliance roles sign off on what ships. Only the release engineer, whose role carries the release permission, promotes it. The agency or consulting team gets a rotatable key, never the root key and never the command line.

Trust setup: decide once, per environment, what each system will believe. Most of the breaches on the timeline were failures of trust, not of encryption. Write the trust list down and keep it under version control:

WHAT IS TRUSTED	SET WHERE	CHANGES ONLY BY
Signing keys that verify mandates and receipts	The published JWKS, with key ids and end dates	Rotation through the ceremony
Certificate roots a program accepts	The trust store in each runtime and container, listed in the SBOM	An updated runtime or bundle, recorded in the SBOM — how the eMudhra roots get removed
Packages and crates	Lockfiles, hash-pinned installs, and `cargo vet` against named published audits	A reviewed audit, never a wildcard
Build tools and CI actions	Commit-hash pins and a read-only job token	A reviewed pull request
People and agents	Roles granted by invitation; agent mandates with caps and expiry	An entitlement change on the ledger
Networks	Client-IP filters on API tokens; Cloudflare Access on admin paths	The key holder

The configurator (Configuration System) shows which environment you are in and which keys you hold. Environment rules, the key-handoff matrix and the RACI are in the Key Management Lifecycle.

Key layers: what each key protects

Not every key is equal. Each layer protects something different, is held by someone different, and costs something different to replace. Knowing which layer a key belongs to tells you who may hold it and what to do the day it leaks.

LAYER	EXAMPLE IN CRM SYNC	WHAT IT PROTECTS	WHO HOLDS IT	IF IT LEAKS
1 · Platform root	`ADMIN_KEY`, and the Cloudflare API token that deploys and sets secrets	Everything: configuration, every tenant, the ability to change the other keys	The key holder only. Never an agency, never CI	Rotate at once through the ceremony, then rotate every key below it
2 · Rotatable admin	A rotatable admin key issued alongside the root	Day-to-day admin work, without exposing the root	The operations team	Rotate it. The root and other teams are untouched
3 · Tenant	`crm_t_` tenant tokens	One store's data and settings — nothing else	That store's team, or its agency	Revoke that one token; other tenants never notice
4 · Sessions	`JWT_SECRET` (HMAC-signed session tokens)	Signed-in sessions: who is logged in right now	The platform, as a secret, never a person	Rotate; every session ends and people sign in again. **Today it also does a second job** — see below
5 · Signing	Ed25519 (EdDSA) keys for entitlements, mandates and receipts; ECDSA P-256 keys for agent-commerce (UCP) request signing, which the UCP standard requires	Receipts, mandates and certificates anyone can check offline; agent requests other platforms can verify	Generated inside the Worker; only the public halves are published (`/.well-known/jwks.json`, `/.well-known/ucp`)	Rotate. Past receipts keep verifying until the old key's end date; new ones use the new key
6 · Provider	Shopify Admin token, Xano API key, Google client secret, payment keys	Access to systems someone else runs	The platform as secrets, set from the provider's dashboard by the key holder	Rotate at the provider first, then store the new value with the ceremony

A correction we found in our own setup. The vault and the encrypted configuration fields are meant to have their own master secret, `ENVELOPE_MASTER_KEY`. When it is not set, the code falls back to `JWT_SECRET` — and in CRM Sync's production it is not set. So one secret is doing two layers' jobs:

- it signs sessions (layer 4)
- it is the root of the encryption keys for vaulted firmware and tenant configuration

Rotating `JWT_SECRET` today would not only sign everyone out; it would also leave every vaulted image and encrypted configuration field unreadable until they are re-wrapped. That is exactly the "one key spanning layers" problem instruction 2 below warns about.

The fix is a planned migration, not a one-line change:

1. Create `ENVELOPE_MASTER_KEY` through the ceremony.
2. Re-wrap every existing envelope under it.
3. Only then leave `JWT_SECRET` to sessions alone.

Setting the new secret without step 2 would lock the existing data at once, because the code would immediately start deriving keys from it. Until the migration is done, treat `JWT_SECRET` as a layer 1 key.

Instructions — run the layers in this order:

1. **List every key you have and give it a layer.** If you cannot say which layer a key protects, it is probably layer 1 by accident. An "all-access" token used for convenience is the common case.
2. **Split anything that spans layers.** One key doing root work and tenant work means a tenant leak becomes a platform leak.
3. **Give each layer its own holder.** Agencies and contractors get layer 2 or 3, never layer 1. Reviewers and AI assistants get none.

4. **Give each layer its own environment copy.** Dev, Stage and Prod never share a value at any layer.
5. **Set a rotation date per layer** — the root on staff change and at least yearly, the others every 90 days and on any incident — and record each rotation by fingerprint.

Without Cloudflare

The ceremony does not need Cloudflare. Wrangler is only Cloudflare's way into its secrets store, and the rule is the same everywhere: **create the secret with OpenSSL and send it straight into the store, through a pipe or a file only you can read.**

PLATFORM	SECRETS STORE	GENERATE AND STORE, NOTHING SHOWN
Google Cloud	Secret Manager	`openssl rand -hex 32 | gcloud secrets versions add NAME --data-file=-`
AWS	Secrets Manager	`( umask 077; openssl rand -hex 32 > ~/.k )`, then `aws secretsmanager put-secret-value --secret-id NAME --secret-string file://$HOME/.k`, then delete `~/.k`
Azure	Key Vault	Same file approach: `az keyvault secret set --vault-name V --name NAME --file ~/.k`
HashiCorp Vault	KV store	`openssl rand -hex 32 | vault kv put secret/app api_key=-`
GitHub Actions	Repository or environment secrets	`openssl rand -hex 32 | gh secret set NAME --env production`
Kubernetes	Secrets, ideally through an external secrets operator	`openssl rand -hex 32 | kubectl create secret generic app --from-file=api_key=/dev/stdin`
Vercel	Environment variables	`openssl rand -hex 32 | vercel env add NAME production`
A server you control	A file only the app's user can read, outside the web root	Over SSH: `( umask 077; openssl rand -hex 32 > /etc/app/api_key )`

Check each command against the provider's current documentation before relying on it.

Shared hosting is the exception. cPanel-style PHP hosting usually has no secrets store at all. Keys end up in `wp-config.php` or a `.env` inside the web root, readable by anything that runs there — including a php2shell exploit. You cannot run a proper ceremony on it. Move secrets outside the web root with `chmod 600` today, and plan the move off it.

The other Cloudflare pieces in this article have equivalents:

THIS ARTICLE USES	WITHOUT CLOUDFLARE
Edge certificates issued and renewed automatically	Let's Encrypt via certbot, Caddy's built-in HTTPS, AWS Certificate Manager, Google-managed certificates
Cloudflare Access on admin paths	Tailscale or a VPN, Google IAP, AWS Verified Access, an IP allowlist
Client-IP filter on the API token	AWS IAM `aws:SourceIp` conditions, Google VPC Service Controls, Azure Conditional Access
WAF rules for php2shell	AWS WAF, Google Cloud Armor, Azure WAF — patching PHP is still the only complete fix

To use the Cloudflare route instead, the free plan is enough to start:

1. **Sign up at dash.cloudflare.com/sign-up.** Workers are on the free plan and run on a `workers.dev` address, so you do not need a domain.
2. **Install Wrangler in your project:** `npm install --save-dev wrangler`. Use Node 24.22.1 or later, which rejects the eMudhra chains.
3. **Sign in from your own terminal:** `npx wrangler login`. This opens the browser and stores a login token on the machine. On a shared machine, run `npx wrangler logout` after the ceremony. For production, prefer an API token scoped to the one account, locked to your network's addresses.
4. **Run the ceremony** above, one environment at a time.

To manage the keys you hold in CRM Sync — create, rotate, revoke — open the Configuration System.

TLS automation: replacing a certificate is a renewal, not a project

Browsers received the eMudhra block automatically. Much other software did not:

- Go binaries, which need a rebuild with a fixed Go
- anything using Python's `certifi` bundle

- the curl that ships with macOS
- older `cloudflared`
- origin servers running their own mutual TLS
- containers with a baked-in trust store

Two habits close that gap:

- **Let the platform issue and renew certificates.** On Cloudflare, edge certificates are issued and renewed automatically. A certificate that chains to a distrusted root is replaced by the next renewal, not by a ticket. A site that buys and installs certificates by hand has until 27 October to get off the emSign hierarchy.
- **Treat the trust store as a dependency, and list it in the SBOM.** `certifi`, `webpki-roots`, the Node and Go versions and the container base image all decide which roots a program trusts. When they are listed, "who still trusts the bad root" is a search across releases.

Python and Docker after eMudhra: what to do this week

Python is the slowest runtime to recover from eMudhra, for a structural reason: **pip carries its own copy of `certifi` and always trusts it, on top of the operating system's list.** There is no setting to turn it off, and `--cert`, `PIP_CERT` and `REQUESTS_CA_BUNDLE` only *add* certificates. A fully patched operating system still leaves an old pip trusting the bad chain. The risk is an intercepted connection serving a fake PyPI, and the only fix is a new pip.

TOOL	FIXED VERSION	NOTES
pip	**26.2.2** or later	The one that matters most: it brings its own certificates
certifi	**2026.10.19**	Removes the emSign roots (advisory GHSA-6q48-6c7w-9gw2)
uv	**0.13.4** or later	Advisory GHSA-hq85-5cpj-mr62; run with system certificates
sigstore, conda, pixi	4.5.1 · 26.9.4 · 0.82.1	—
Node	22.23.4 · 24.22.1 · 26.12.1	Same releases as the Brotli fix

Versions are from the Python packaging announcement on discuss.python.org.

Upgrade pip from the pinned wheel, not with a bare `pip install -U pip`. An unpatched pip fetching its own replacement is exactly the connection an attacker would intercept. The announcement gives a `pip install` command naming the 26.2.2 wheel by its SHA-256 hash, so a substituted file is refused. Then upgrade `certifi` and `uv`. This is the hash-pinned install from the supply-chain table, doing the job it exists for.

Every virtual environment, not just the system Python. Each venv has its own pip. A **new** venv still gets the old bundled pip from `ensurepip` until CPython ships its fix (GH-159910). So upgrade pip right after creating any venv, and in CI after every `python -m venv`.

Docker: a patched host does not patch an image. The certificates are baked in at build time.

- **Official `python` images:** the maintainers rebuilt them on 21 October (docker-library/python #1142), with the expedited 3.13.17, 3.14.9 and 3.15.1 releases. The **3.11 and 3.12 images keep an older pip**, because upstream does not change pip in those security releases. Add the pinned-wheel pip upgrade to your Dockerfile, or move to 3.13 or later.

- **Rebuild without the cache**, so the new base and certificates are actually used: `docker build --pull --no-cache`. A cached layer quietly keeps the old trust store.
- **Node and Go images on Debian** also need `ca-certificates` updated inside the image, so curl, wget and git stop trusting the chain.
- **List the trust store in the SBOM** (pip, certifi, `ca-certificates`, base-image digest), so "which images still trust emSign" is a search, not an audit.

And the LiteLLM root cause shows why the key ceremony keeps the AI out. On 22 October LiteLLM disclosed how 1.107.0 was published:

- **The tokens were in logs.** An internet-exposed internal LiteLLM deployment held GitHub CLI OAuth tokens. LiteLLM says an AI coding tool's response bodies can carry the user's GitHub CLI token, and those bodies were logged.
- **The attacker used them to publish.** With three developers' tokens, including one with workflow rights and an organisation owner's token used to switch off branch protection, the attacker pushed the release through a lookalike repository. It was signed and published like a real release.
- **The stolen cloud keys were used within hours.** Stolen AWS credentials were being tested about four and a half hours later.
- **Release rights had just been widened.** LiteLLM had temporarily expanded them for two engineers two days earlier.

Every link in that chain is covered in this article:

- AI traffic carries secrets, so gateway logs must not store request and response bodies.
- Developer tokens should be scoped to one repository and to read access unless a release needs more.
- Owner tokens should not sit on everyday machines.
- Release rights widened "temporarily" need an end date, like any other key.

The order: fix trust first, then rotate keys

New keys do not fix eMudhra. The two problems sit at different layers:

	WHAT IT PROTECTS	WHAT FIXES IT
eMudhra	Whether a computer can tell the real pypi.org — or any site — from a fake one	Updating what the software trusts: pip 26.2.2, certifi, uv, Node, `ca-certificates`, rebuilt images
Your keys (Wrangler secrets)	Your own secrets: admin keys, webhook secrets, provider API keys	The key ceremony

A forged pypi.org is accepted before any of your keys is involved, so rotating them cannot stop it. The order:

1. **Fix trust first.** Update Node, pip (from the hash-pinned wheel), certifi and uv on every machine and in every venv, and rebuild images. This is the eMudhra fix.
2. **Then ask whether anything leaked while you were exposed.** If a `pip install`, a `wrangler secret put` or a deploy ran over an untrusted network while the old trust was in place, a secret could have crossed an intercepted connection. Rotate those secrets with the ceremony, **from the patched machine**.
3. **If nothing points to exposure, rotate on the normal schedule.** eMudhra alone does not require new keys.

The ceremony is the clean-up and the habit, not the cure. Run on an unpatched machine, it would send fresh keys over the same untrusted connection it is meant to protect.

Clean up after exposure: rebuild, don't purge

A package downloaded through an intercepted connection has the right name and the right version, and nothing on disk records that its download was faked. So you cannot find the bad ones by looking, and uninstalling packages

one at a time does not work. **Rebuild instead, from hashes recorded before the exposure.**

1. **Fix pip first**, as above: 26.2.2 from the hash-pinned wheel, then certifi and uv, on a network you trust. A new Python alone does not fix existing environments, because every venv keeps its own pip.
2. **Delete each virtual environment and recreate it from a lockfile with hashes:** `pip install --require-hashes -r requirements.txt`, or `uv sync` from a `uv.lock`. Every package is checked against its recorded hash, and a substituted file is refused. No lockfile yet? Generate one with the patched pip, then rebuild from it.
3. **Clear every cache a tampered file could hide in**, or the rebuild reuses it:
   - pip: `pip cache purge`
   - uv: `uv cache clean`
   - Docker: `docker build --pull --no-cache`
   - any shared wheelhouse or internal package mirror
4. **Rotate secrets only where there is a reason.** A malicious package runs code while it installs, so removing it afterwards does not undo what it read. If installs ran over an untrusted network while the old pip was in place, rotate what those environments could reach, using the ceremony and the patched machine. If every install ran on networks you trust, rebuilding is enough. No attack has been reported.
5. **Scan what you rebuilt** against published advisories: `pip-audit`, or `osv-scanner` across lockfiles and images. A scanner finds known-bad versions, not swapped files. Steps 2 and 3 handle swapped files.

The exception is LiteLLM 1.107.0. That release was malicious, not merely exposed. Anywhere it was installed:

- treat the environment as compromised

- do not start Python there to clean up, because starting Python re-runs the payload
- rebuild from 1.106.2 pinned by digest
- rotate everything that environment could reach

Does `cargo vet` help? For Rust, yes, but at a different layer from eMudhra:

- **Against an intercepted download:** Cargo already checks every crate against the checksum in the committed `Cargo.lock`. A Rust build from a lockfile was protected the way a hash-pinned pip install is. Keep the lockfile committed and build with `--locked`.
- **Against a poisoned release** — the LiteLLM kind, published by a stolen account: this is what `cargo vet` is for. CI fails on any crate version that no trusted, published audit covers, so a hijacked new release is stopped before anyone has reported it. Neither checksums nor scanners can do that, because the bad release is "genuine" when it ships.
- **For eMudhra itself:** Rust programs that use `webpki-roots` need its fixed release and a rebuild. `cargo update -p webpki-roots`, then `cargo vet` asks for that new version to be audited or certified before it ships. That is the review step working as intended.

Python has no direct equivalent of `cargo vet` yet. The nearest substitutes:

- hash-pinned lockfiles
- a delay before adopting new versions
- review of each dependency change in the pull request

That is the Python row of the supply-chain table above.

AI binding: the agent proves its permission on every call

An AI agent in this architecture acts under a signed mandate: who may act, on what, up to what amount, and until when. The mandate is checked against the

published key on every call, so:

- an agent holding a mandate signed by a retired key is refused
- a mandate altered after signing fails verification
- a stolen environment yields keys that have already rotated

Bound tasks and TLS hops are covered in Identity, data and AI tasks. Signing is what lets a third party — an auditor, a partner platform, Google — check that work without trusting us.

Records on a timeline that scales out

Every grant, check, firmware download and consent change is appended to a hash-chained ledger. Each entry locks in the one before it, so history can be visibly broken but never quietly edited. The ledger is not one long chain. **Each tenant and each stream is its own thread**: firmware for one product, mandates for one agent, consent for one store.

- **Threads do not wait on each other.** A new store or product line adds a thread, not contention on a shared one, so the record scales horizontally.
- **Order within a thread is set by a sequence number**, not a clock. Two simultaneous writes cannot be mistaken for tampering.

On a breach day, the incident dates in the table above are laid over those threads. "Everything signed by key X between 30 April 2025 and 17 October 2026" or "every image served with an SBOM listing `certifi` older than the fix" becomes a filter, not an investigation. To see one session's thread across consent, identity, purchases and engagement, open **See a session's story** on the verify plane.

The benefit, control by control

CONTROL	BREACHES IT ANSWERS	BUSINESS BENEFIT
SBOM for every release (CycloneDX or SPDX)	php2shell, Node, LiteLLM, eMudhra trust bundles	"Are we affected?" answered in minutes, and the record EU Cyber Resilience Act reporting asks for
Signed firmware and build images in a vault	A swapped or poisoned image in distribution	Distributors and devices check the hash against the certificate; a leak is stopped by one rotation, with no recall
Signing keys with key ids, rotation and end dates	eMudhra, the LiteLLM key theft	A leaked key's useful life ends at rotation; receipts it already signed keep verifying
Automated TLS issuance, with the trust store in the SBOM	eMudhra	A distrusted chain is replaced at renewal, and lagging runtimes are found by search
AI binding to signed mandates	Agent overreach, stolen agent credentials	Every agent action can be checked by a third party without trusting the operator
Threaded, hash-chained ledger	All of them	A timeline that grows with the business and answers incident questions by filter

What signed records do not do. An SBOM does not patch anything. A signature proves who sealed a file, not that the file is safe. A ledger shows that history was broken; it does not prevent the break. Developer machines and CI runners — where Node, Wrangler and Python run — still need the runtime updates in the timeline. More background: What Is an SBOM? Who Uses This? Everyone. and Firmware, SBOM & the Cyber Resilience Act.

Its role in AI-first resilience: for QA and growth marketing teams

AI agents and answer engines now act on a store's behalf and decide what customers see. A journey that quietly breaks — a consent banner, a checkout step, a tracking tag, a product feed — costs sales before anyone notices. The verify plane gives the two teams who catch those breaks the same signed record to work from.

QA and testing teams

CHALLENGE	SOLUTION	BUSINESS OUTCOME
Release sign-off waits on manual checks across regions, browsers and app versions	Every test session leaves a signed record of each consent, identity, cart and order step, open on the verify plane as **See a session's story**	Faster sign-off, with evidence attached instead of screenshots
"It passed on my machine": a failure can't be reproduced or proven	The session's thread shows exactly which step changed, in order, and cannot be edited after the fact	Defects are reproduced from the record, not argued over
AI agents place orders and refunds faster than anyone can watch	Each agent action carries a signed mandate with a scope, a limit and an end date; tests confirm an agent is refused outside it	Agent behaviour is tested like any other feature before it reaches customers
A dependency or certificate change — like eMudhra — breaks a journey after release	Trust stores and packages are listed in the SBOM, so a test run names the release that changed	Regressions are traced to a release in minutes

AEO and tracking growth marketing teams

CHALLENGE	SOLUTION	BUSINESS OUTCOME
Tracking drifts after site changes, and the report is wrong before anyone notices	Each event carries its consent state and is checked against the signed record	Numbers the team can defend in a budget meeting
Consent rules remove data, and nobody can tell lost data from broken tags	The record shows which events were withheld by consent and which never fired	Consent loss is measured, not mistaken for a tracking bug
AI answer engines cite competitors, or show stale prices and stock	Product, price and review data go out as structured data and through the Merchant API from one source of truth	Accurate answers in AI search, and more citations
Attribution stops at the first redirect or app hand-off	One session thread runs from first touch through consent, sign-in and order	Spend follows what actually converts

No silver bullet: rotate, randomise, look forward

No single control in this article would have stopped every incident in it. TLS did not stop the checkout hijack. MFA did not stop nOAuth. A WAF does not stop a poisoned package, and a package audit does not stop an unverified sign-up. What holds is layers that change faster than an attacker can map them — so the defence has to move on a schedule, and be tested in ways the attacker cannot predict.

PRACTICE	WHAT IT TAKES AWAY FROM THE ATTACKER	CADENCE
Rotate every credential — on a schedule, and immediately on any incident, through a recorded key ceremony	The useful life of a stolen key	Every 90 days, and on event
Short-lived sessions, revocation checked on every read	Replaying a captured session	Every request
Keys generated per deployment, never copied between environments	One leak opening every environment	Every deployment
Unguessable identifiers — random, not sequential — in anything a public response returns	Enumerating accounts, orders and records by counting	Always
A fresh Content Security Policy nonce per response	Reusing an injected script	Every response
Testing the attacker cannot predict: rotating penetration-test scope, AI-assisted scanning for chained attacks, canary credentials that alert the moment anyone uses them	Mapping a defence that never changes	Continuous
A delay and an audit gate on every dependency	The first hours of a malicious release	Every update

Build to the standard that is coming, not the one being enforced today. Every requirement in the table at the top of this article was announced years before it was enforced, and the merchants hurt worst in October 2026 were the ones still meeting the old version. The ones already on the calendar: the Content API for Shopping shut down on 18 August 2026; EU AI Act transparency obligations apply from 2 August 2026; the EU Cyber Resilience Act's reporting obligations began on 11 September 2026, with full obligations from 11 December 2027; GS1 Sunrise 2027 moves retail point of sale to 2D barcodes. Each date, with its source, is in The Compliance Calendar.

What it costs

COST	WHAT IT MEANS	SEVERITY
Rework of vendor plugins	Loyalty, upsell and options apps are rebuilt as Functions, Workers and a ledger, not configured	High — weeks per plugin, not hours
A monthly bill above shared hosting	A private VM, Cloudflare plans or managed Kubernetes cost more than a shared box	Low to high by tier; get current quotes
EmDash maturity	A v0.1.0 preview with a near-empty plugin ecosystem; WordPress's co-founder argues the isolation only holds on Cloudflare	Medium — pilot only
MySQL has no row-level security	WooCommerce and Magento data needs a move to Postgres or filtered views	Medium
New dependency ecosystems	Rust, Python and npm each need their own gate	Medium — the gates are the cost
Familiar tooling retired	Teams lose cPanel, FTP and plugin-store installs	Real, and the point

> **What this does not claim.** It is not a certification and not legal advice. Delisting counts come from public trackers, not Shopify. "Unconfirmed" means exactly that. Prices are deliberately absent: they vary by provider, region and traffic.

What to do next

Ordered by exposure — card data first.

1. **Payment pages.** Inventory every script, remove every client-side redirect, set the CSP, run a private-window test checkout. *Owner: store owner with the developer.*
2. **Delisted and hijack-listed apps.** Remove or replace; revoke their Shopify scopes after cutover. *Owner: store owner.*
3. **PHP runtime.** Patch for php2shell and keep the WAF rules in Block. *Owner: hosting or DevOps.*
4. **Origin.** Cloudflare in front, Full (strict), Access on every admin path, tunnel, inbound ports closed. *Owner: DevOps.*
5. **Containers.** One pinned image per app, secrets injected at runtime. *Owner: developer.*
6. **Identity and data.** `/auth/me` sidecar; a database role per service; RLS or filtered views; personal fields encrypted with a key generated for this deployment. *Owner: developer with compliance.*
7. **Checkout logic.** Scripts and checkout.liquid patterns rebuilt as Rust Shopify Functions. *Owner: Shopify developer.*
8. **Google.** Merchant API migration with loyalty, reviews, comparison and conversions in the same pass. *Owner: marketing operations with the developer.*
9. **Content.** Static publish first, EmDash pilot second. *Owner: content lead.*

0. **Decommission** the shared VM after the tunnel has carried production traffic for a full billing cycle. *Owner: DevOps.*

Evidence basis: public vendor, platform and standards documentation linked inline, read on 22 October 2026. Not a certification.
